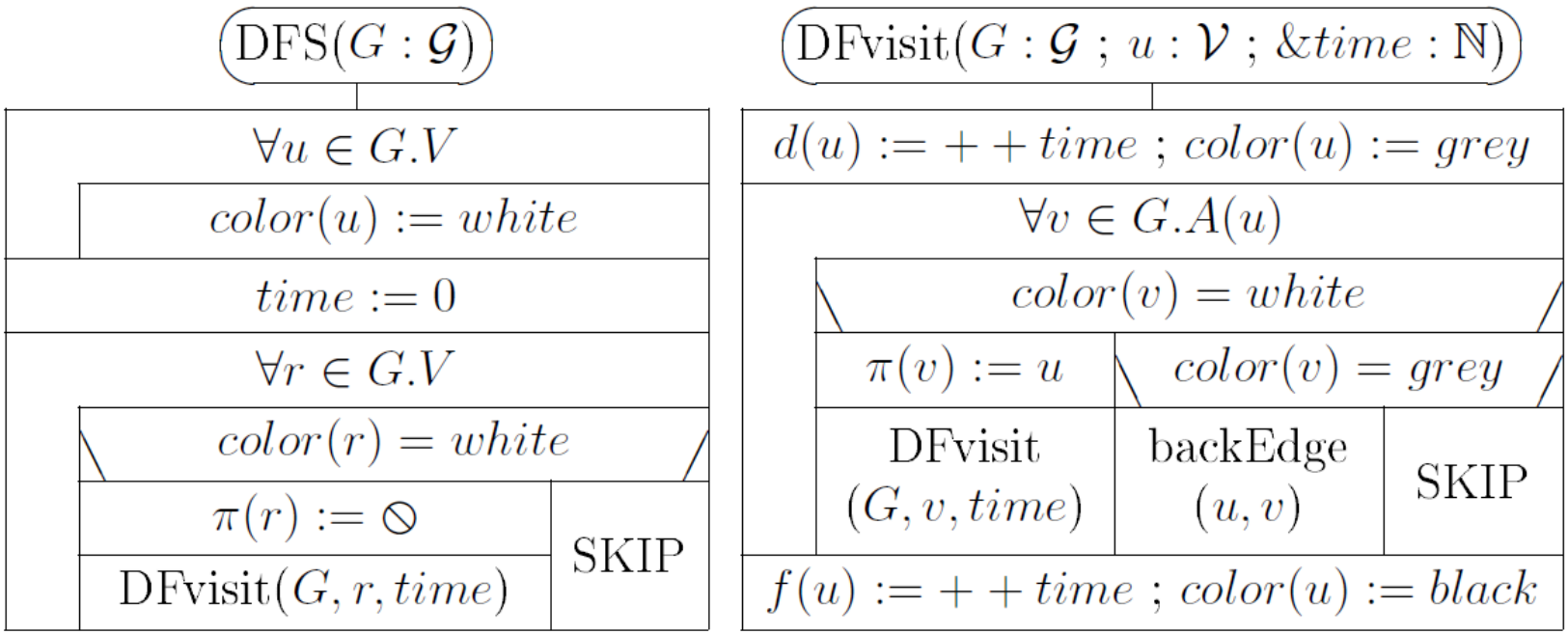
Algorithms and Data Structures 2.
Depth First Search – DFS

Harrach Nóra - harrachnora@inf.elte.hu

2024. szeptember 30.

Depth-First Search (DFS)



- In these lecture notes we consider DFS on digraphs only.
- Unlike in BFS, in DFS the colors of the vertices are essential.
- Unlike BFS, DFS goes on in one direction in the graph while it finds undiscovered, i.e. white vertices.
- When DFS discovers a vertex, it becomes grey.
- When DFS does not find any white vertex as a child of the actual vertex, it colors the actual vertex black, and backtracks to its parent, i.e. to the vertex it was discovered from.
- Then this parent becomes the actual vertex again.
- And DFS tries to go through another, still unprocessed edge to another white vertex, and so on.

DFS is highly non-deterministic: it may select any unprocessed edge going out from the actual vertex. While illustrating DFS, we will resolve this non-determinism: we prefer the edge going to the vertex with the lowest index.

The classical form of DFS does not have any source vertex. The depth-first traversal of the graph consists of depth-first visits (DFvisits). A DFvisit may start from any white vertex of the graph. (While illustrating DFS, we will resolve this non-determinism: we prefer the white vertex with the lowest index.)

Each DFvisit builds up a depth-first tree with its starting point as root. In a depth-first tree, the parent of a non-root vertex is the vertex it was discovered form.

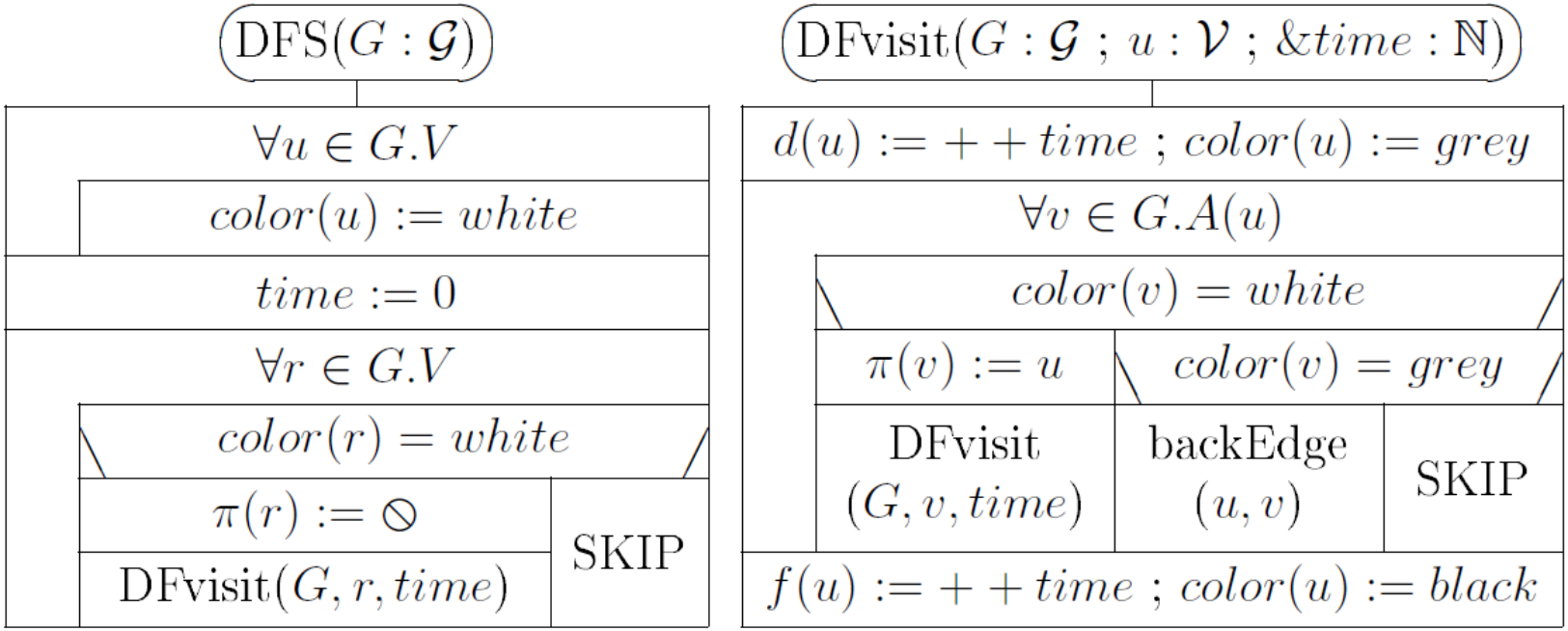
A DFvisit ends when we color its root black, and we cannot backtrack from it, because it does not have any parent. Then we try to start another DFvisit form a white vertex. When no white vertex remains, DFS stops.

At the beginning of the program all vertices are white.
During a DFvisit some vertices can be grey.
Before starting a new DFvisit all the vertices are white or black.
At the end of the program all of them are black.

DFS also has variable time : $\mathbb{N}$. It starts from zero and it is increased when a vertex is discovered or finished.

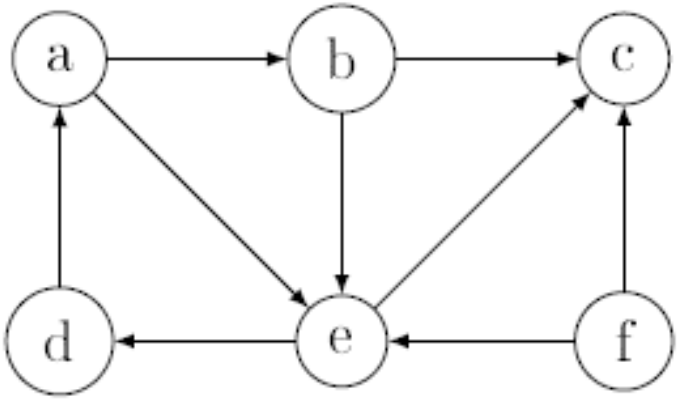
For each vertex of v of the graph, DFS assigns value to the following labels of v.

- color(v) $\in$ {white, grey, black} where color(v) = white means that v is still undiscovered, i.e. no DFvisit has touched it; color(v) = grey means that v has been discovered, but it is still unfinished, i.e. DFS still has not tried to backtrack from it; and color(v) = black means that v has been finished.
- $d(v) \in \{1, 2, 3, ...\}$ is the discovery time of v.
- $f(v) \in \{2, 3, 4, ...\}$ is the finishing time of v.
- $\pi(v) \in V$: is the parent vertex of v and $\pi(v) = \emptyset$ means that v does not have parent (i.e. it is the root of a DFvisit).

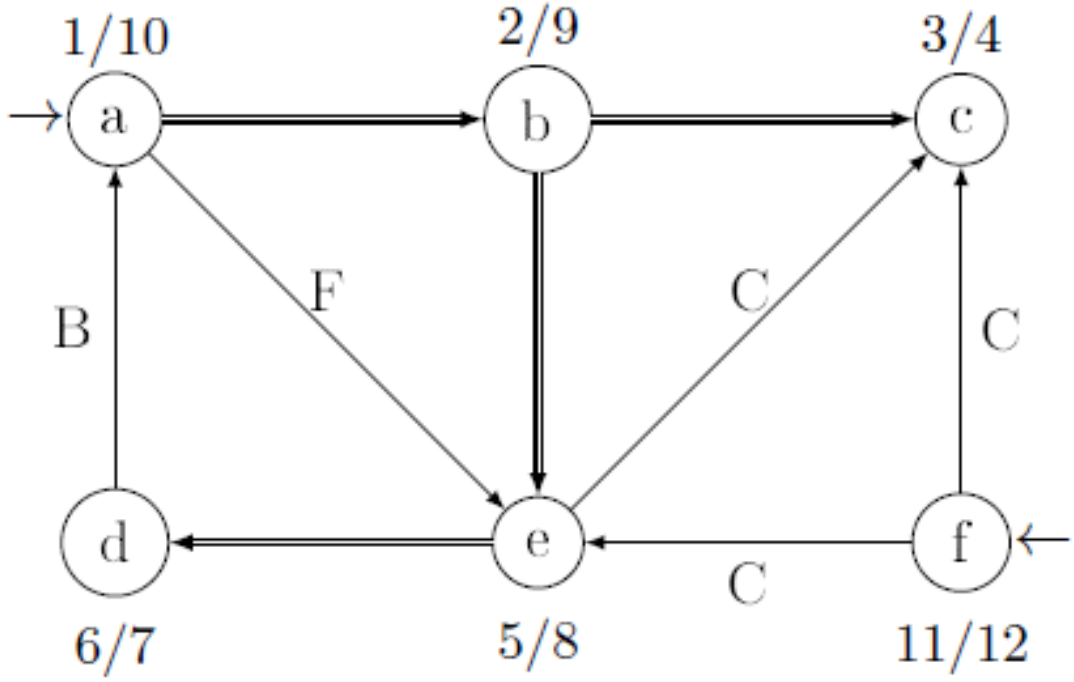


Each DFvisit (started from DFS) computes a depth-first tree.
The depth-first forest consists of these depth-first trees.
$r \in G.V$ is the root of a depth-first tree $\iff \pi(r) = \emptyset$.
$(u, v) \in G.E$ is the edge of a depth-first tree $\iff u = \pi(v)$.

a → b ; e.
b → c ; e.
d → a.
e → c ; d.
f → c ; e.



DFS on this digraph:



Running time of DFS

Procedure DFS is called only once. Both loops of DFS iterate n times, so we have $2n + 1$ steps without DFvisits.

Procidure DFvisit is also called n times: once for every vertex at the point this vertex is colored from white to grey. The loop of DFvisit processes the edges of the graph, each edge is processed by one iteration, so we have m iterations.

We suppose here that a call to procedure backEdge just labels a back edge, so its operational complexity is $\Theta(1)$.

We have counted $n + m$ steps in DFvisits, so $3n + m + 1$ steps alltogether, from which

$$MT(n), mT(n) \in \Theta(n + m)$$

Classification of edges:

- (u, v) is a tree edge $\iff$ (u, v) is the edge of some depth-first tree (denoted by strong/double line)
- (u, v) is a back edge $\iff$ v is an ancestor of u in a depth-first tree (denoted by B)
- (u, v) is a forward edge $\iff$ (u, v) is not a tree edge, but v is a descendant of u in a depth-first tree (denoted by F)
- (u, v) is a cross edge $\iff$ u and v are vertices on different branches of the same depth-first tree, or they are in two different depth-first trees (denoted by C)

Recognizing the edges of a graph:

When we process edge (u, v) during a DFvisit, this edge can be classified according to the following criteria.

- (u, v) is a tree edge $\iff$ vertex v is still white.
- (u, v) is a back edge $\iff$ vertex v is grey.
- (u, v) is a forward edge $\iff$ vertex v is already black and $d(u) < d(v)$.
- (u, v) is a cross edge $\iff$ vertex v is already black and $d(u) > d(v)$.

Checking the DAG property of a graph

Definition. A directed graph G is a DAG (Directed Acyclic Graph) $\iff$ it does not contain a directed loop.

When DFS finds a back edge (u, v), then we have found a directed loop in the graph.

Why? Because v is an ancestor of u in the depth-first tree which is being built, so the path consisting of the tree edges from v to u, together with the edge (u, v) form a directed loop.

Theorem. A directed graph G is a DAG $\iff$ DFS does not find a back edge.
If DFS finds a back edge (u, v), then the sequence of vertices $< v, u, \pi(u), \pi(\pi(u)), \ldots, v >$ read backwards form a simple directed loop.

Homework:

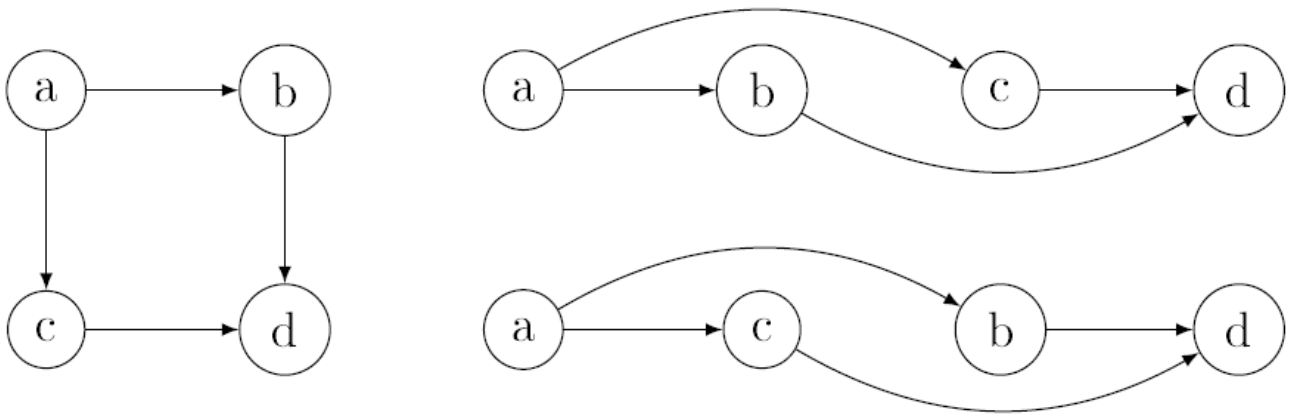
1. Write procedure backEdge(u, v) which prints all loops when a back edge is found. Make sure the time complexity is $O(n)$.
2. Modify DFS so that when we use it on **undirected graphs** to find loops.

Topological sort

Definition. A topological order of a digraph G is a linear ordering of all its vertices such that for every edge (u, v) it is true that u is before v in this ordering.

A graph may have more than one topological order.

Example of a graph and its two different topological orders:



Theorem. A digraph has topological order $\iff$ it is a DAG.

Definition. A graph aglorithm which generates a topological order in a DAG is called a *topological sort.*

We can perform a topological sort along with DFS: consider the order of vertices acording to the finishing times of DFS. Reverse this order to obtain a topological order.

Topological sort of a DAG with DFS:

1. Create an empty stack.
2. Perform DFS on the digraph. Every time a vertex is finished, put it on the top of the stack.
3. If DFS finds a back-edge, then the graph is not a DAG, there is no topological ordering, and the content of the stack is undefined.
4. Otherwise, the final content of the stack in top-down order is the topological order of the DAG.

Homework. Write the structograms of (1) DFS and (2) topological sort in cases of (A) adjacency list and (B) adjacency matrix representations of the graph. What can you say about the efficiencies of the four implementations?